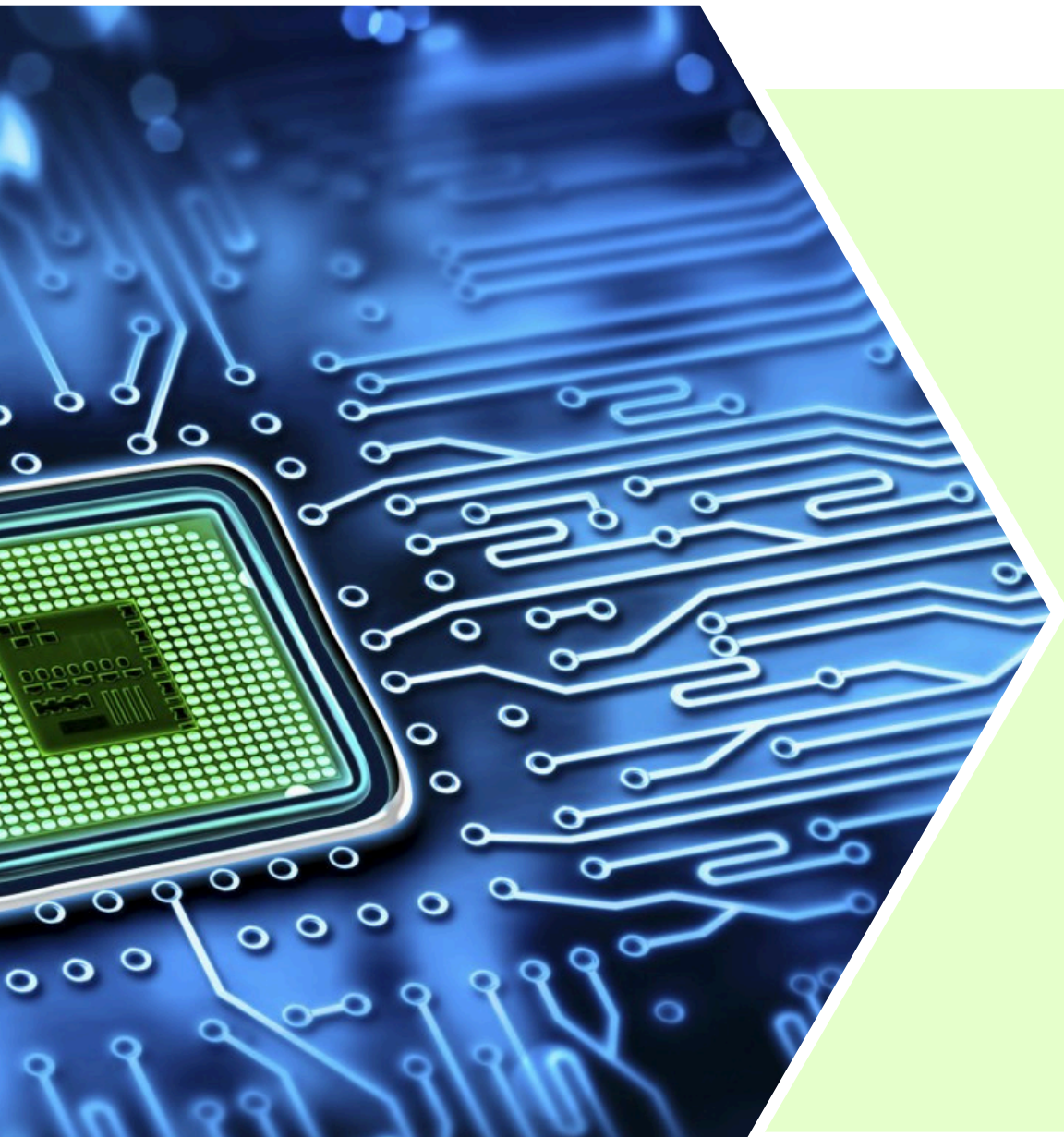


SMART REACTIVE AUDIO-VISUAL SYSTEM



TEAM MEMBERS

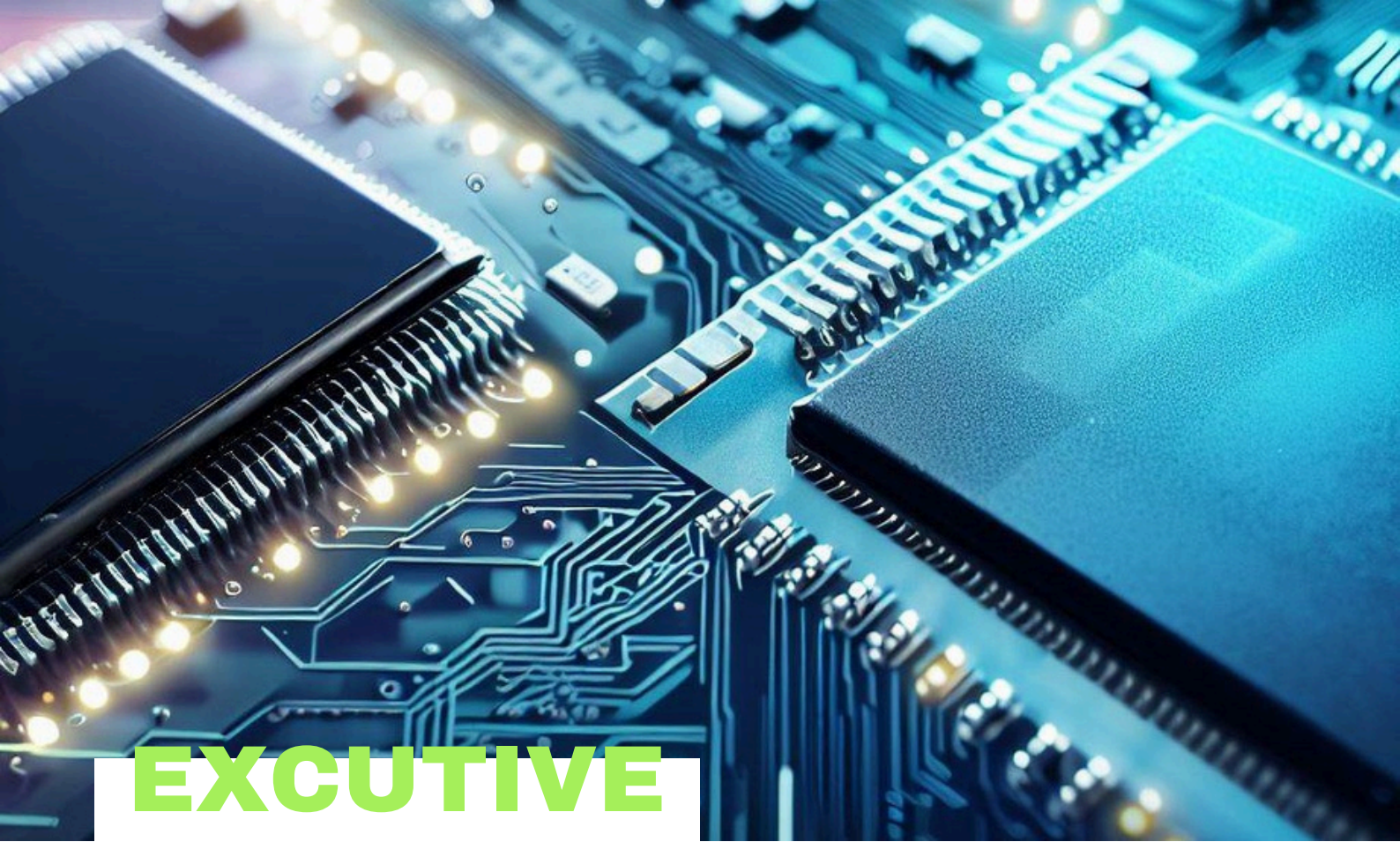


- هنا أحمد نبهان
- اية محمد سيد
- هاجر علي الرفاعي

Submitted To:

DR: Lamiaa Elrefaie

ENG ./ MAHMOUD NAWAR



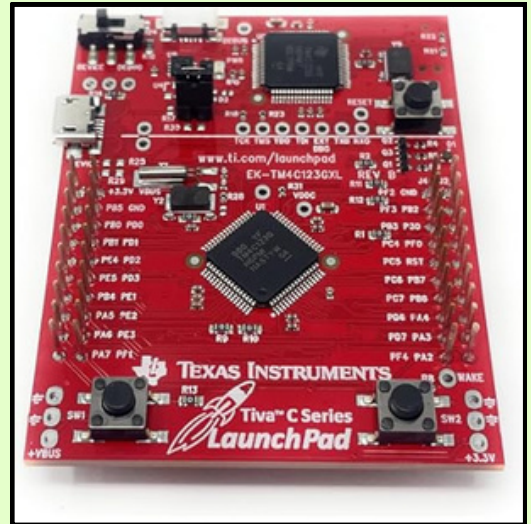
EXECUTIVE SUMMARY

This project involves designing and implementing a Smart Reactive Audio-Visual System using the Tiva C microcontroller. The system provides a combination of audio and visual feedback by controlling an LED light strip that reacts to sound intensity . The system also includes two switches to control the operation of the LED lights and an LCD. Various hardware peripherals, interrupts, and communication protocols are utilized to achieve real-time interactivity.

System Overview

A Tiva C Microcontroller

Serves as the central controller for managing all components and providing real-time functionality..



B LED Light

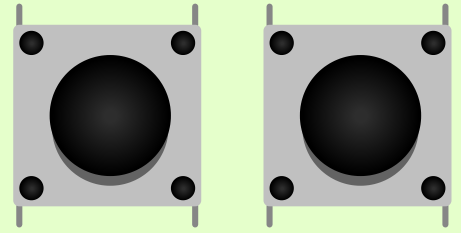
Reacts dynamically to the intensity of sound detected by a microphone.



System Overview

C

Two Switches:



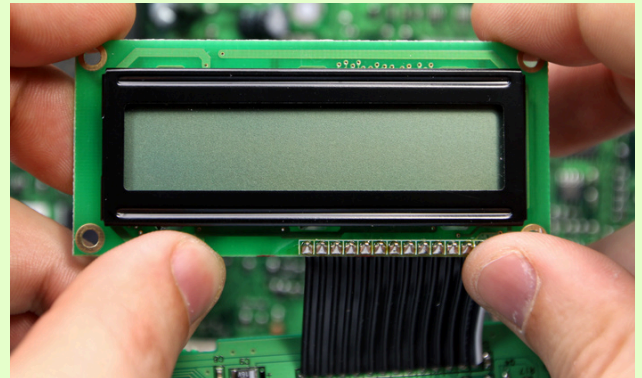
- Switch 1: Toggles the LED light ON/OFF.

Switch 2: Toggles the LCD ON/OFF.

D

LCD Display

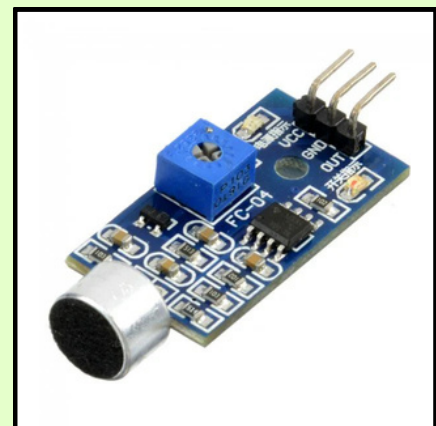
Displays sound intensity and slider values.



E

Microphone

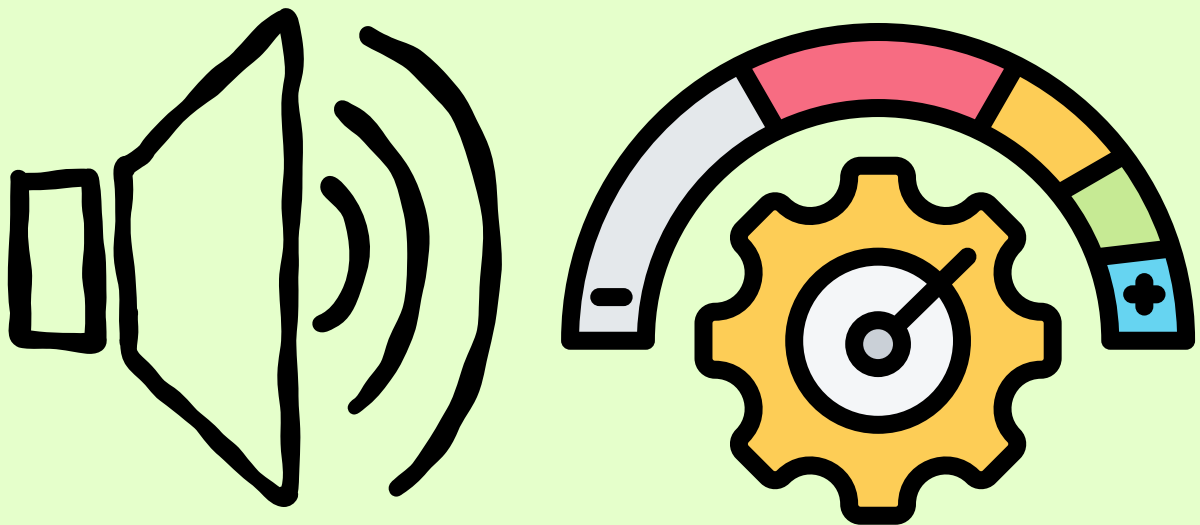
Detects sound intensity using an analog sensor.



System Functionality

A Sound Intensity Detection:

The microphone module detects sound intensity, converts it to an analog signal, and feeds it to the Tiva C's ADC. The ADC converts the signal into a digital value, which is used to control the brightness of the LED lights.



C LCD Display:

The LCD displays real-time sound intensity and potentiometer values for user feedback.

Drivers Used

A GPIO DRIVER

This GPIO driver provides essential functionality to configure and control GPIO pins. By abstracting the low-level register operations into intuitive function calls, it simplifies GPIO management for embedded systems development. Each function targets a specific aspect of GPIO configuration, ensuring modular and maintainable code.

1. Static Array Declaration:

This array provides an indexed way to access the GPIO port registers (GPIOA, GPIOB, etc.) by a character identifier for the port (e.g., 'A', 'B', etc.)

- `GPIOA_Type*`: A pointer to a structure representing the memory-mapped GPIO peripheral.
- `static`: Limits the scope of this array to the file where it is defined.
- `const`: Indicates the array elements are constant pointers (they cannot be changed to point elsewhere).

Drivers Used

2. GPIO_EnableClock Function

Enables the clock for the specified GPIO port.

- **Parameters:**
- **port:** The character identifying the port (e.g., 'A' for GPIOA).
- **Functionality:** This typically involves setting a bit in the microcontroller's RCC (Reset and Clock Control) peripheral register to enable the clock for the chosen GPIO port.

3.GPIO_Init Function:

Initializes a specific GPIO pin with the given direction and mode.

- **Parameters:**
- **port:** GPIO port identifier.
- **pin:** GPIO pin number.
- **dir:** Direction (e.g., input or output).
- **mode:** Pin mode (e.g., push-pull, open-drain, alternate function, etc.).

Drivers Used

4. GPIO_SetDirection Function:

Sets the direction (input or output) of a GPIO pin.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.
- dir: Direction (e.g., '0' for input, '1' for output).

5. GPIO_analog_digital Function:

Initializes a specific GPIO pin with the given direction and mode.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.
- dir: Direction (e.g., input or output).
- mode: Pin mode (e.g., push-pull, open-drain, alternate function, etc.).

6. GPIO_ReadPin Function:

Reads the current logic level of a specific GPIO pin.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.
- Returns: The logic level (e.g., '0' or '1') of the pin.

Drivers Used

7. GPIO_WritePin Function

Sets the output logic level of a GPIO pin.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.
- value: Logic level to write (e.g., '0' or '1').

8. GPIO_EnablePullUp Function:

Enables the pull-up resistor for a GPIO pin.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.

9. GPIO_MODE_SELECT Function:

Configures the mode of a GPIO pin.

- Parameters:
- port: GPIO port identifier.
- pin: GPIO pin number.
- mode: Mode (e.g., input, output, alternate function, etc.).

Drivers Used

B LCD DRIVER

The LCD driver provides functions to control a 4-bit mode LCD, enabling efficient communication and display management. Key functionalities include initialization, data/command writing, cursor movement, and screen clearing.

1. LCD4bits_Init Function

Initializes the LCD in 4-bit mode.

- Configures the GPIO pins connected to the LCD (data pins, RS, RW, and EN) as outputs.
- Sends initialization commands to set up the LCD in 4-bit mode, configure the display, clear the screen, and set the cursor's initial position.

2. LCD_WriteString Function

Writes a null-terminated string (\0) to the LCD.

- Parameters:
- str: A pointer to the string to be displayed on the LCD.

Drivers Used

3. LCD_Write4bits Function

Sends 4 bits of data or a command to the LCD.

- **Parameters:**
- **data:** The 4-bit data to be written to the LCD.
- **control:** Indicates whether the data is a command (0) or a character (1)

4. LCD4bits_Cmd Function

Sends a command to the LCD.

- **Parameters:**
- **cmd:** The command to be sent.

5. LCD4bits_Data Function

Writes a single character to the LCD.

- **Parameters:**
- **data:** The character (ASCII value) to be written.

Drivers Used

6. LCD_vCLR Function

Clears the LCD screen.

- **Functionality:**
- Sends the 0x01 command to clear the display.
- Resets the cursor position to the top-left corner.

7. LCD_vMoveCursor Function

Moves the LCD cursor to a specific position.

- **Parameters:**
- row: The row number (1 or 2 for a 16x2 LCD).
- column: The column number (1 to 16 for a 16x2 LCD).



GPIO INTERRUPTDRIVER

This GPIO INTERRUPT driver provides functionality to configure and handle interrupts for GPIO pins, enabling event-driven programming in embedded systems.

Drivers Used

1. INTERRUPT_Init Function

This function configures Port C (pins 4 and 5) for edge-triggered interrupts on falling edges.

Steps:

1. Enable Clock for Port C
2. Set Pins 4 and 5 as Input
3. Enable Digital Functionality
4. Enable Pull-Up Resistors
5. Configure Edge-Sensitive Triggers (Edge-sensitive - Not both edges - Falling edge)
6. Clear Pending Interrupt Flags
7. Enable Interrupts for Pins 4 and 5
8. Set Interrupt Priority and Enable in NVIC
9. Enable Global Interrupts

2. GPIOPortF_Handler

Handles the interrupts for switches connected to PC4 (Switch 1) and PC5 (Switch 2).

- Check Interrupt Source on RIS register
- Acknowledge Interrupt on ICR register
- Toggle State for Switch 1
- Handle PC5 Similarly

Drivers Used

3.Switch State Functions

These functions return the current state of the switches

- `readState1` ---> Returns the state of Switch 1
- `readState2` ---> Returns the state of Switch 2

D TIMER DRIVER

This TIMER driver provides precise timing operations, such as delays, periodic interrupts, or PWM generation.

1.Timer2_Init

Parameters

- `period`:
 - Type: unsigned long
 - Specifies the timer period in clock cycles.
 - The value depends on the clock frequency and the desired timer duration.
 - Example: If the system clock is 16 MHz, a period of 16000 corresponds to a 1 ms timer.

Drivers Used

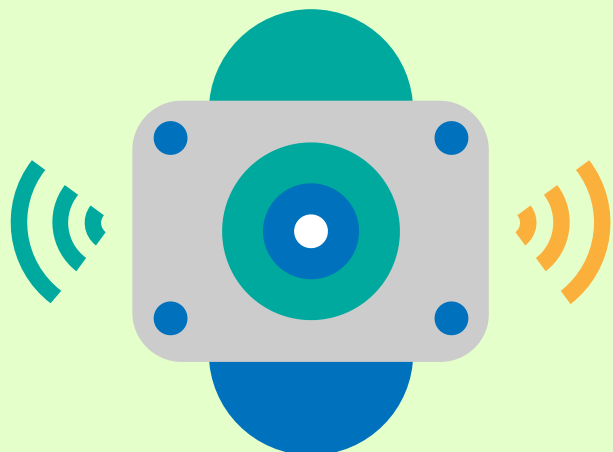
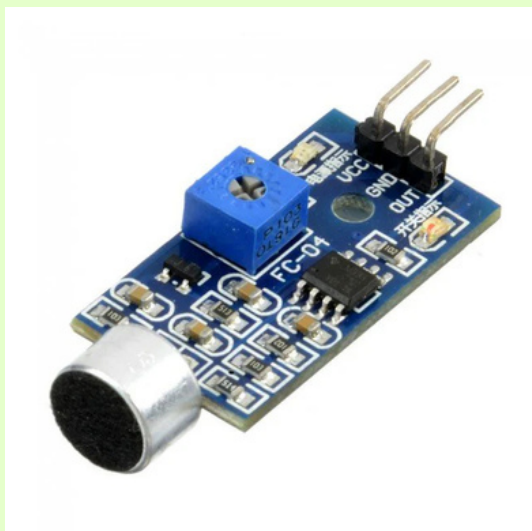
F ADC DRIVER

ADC (Analog-to-Digital Converter) driver for the TM4C123 microcontroller, which sets up ADC0 to read analog values from channel 0 (pin PE3). The ADC value is processed using sequencer 3, which handles one sample at a time.

1. volatile unsigned int ADCvalue;

Stores the digital representation of the voltage from the analog pin (PE3).

- Declared as volatile because it is modified inside an interrupt handler and accessed outside the handler.



Drivers Used

2. ADC0_Init_Ch0

This function initializes ADC0 to read analog values from PE3 (AIN0).

1. Enable Clock to GPIO Port E and ADC0
2. Configure PE3 as Analog Input
3. Configure ADC Sample Sequencer 3
4. Enable Interrupts for ADC
5. Enable and Start ADC

3. ADC0Seq3_Handler

The interrupt handler for ADC0 sequence 3.

- Read ADC Conversion Result
- Clear the Interrupt Flag
- Start the Next Sampling

4. ADC0_Read Function

This function returns the latest ADC value read from PE3.

Drivers Used



LED DRIVER

simple interface to control LEDs using GPIO pins. It consists of three main functions for initialization, turning the LED on, and turning it off.

1. LED_voidINIT Function

This function initializes the specified GPIO pin for the LED as an output and sets it to operate in digital mode.

2.LED_voidON

This function writes a logic HIGH (1) to the specified GPIO pin, turning the LED on.

3.LED_voidOFF

This function writes a logic LOW (0) to the specified GPIO pin, turning the LED off.

Drivers Used

H SWITCH DRIVER

This driver provides an interface for working with buttons connected to GPIO pins. It includes functions for initializing a button and reading its state.

1. BUTTON_voidINIT Function

This function prepares the specified GPIO pin for use as a button input.

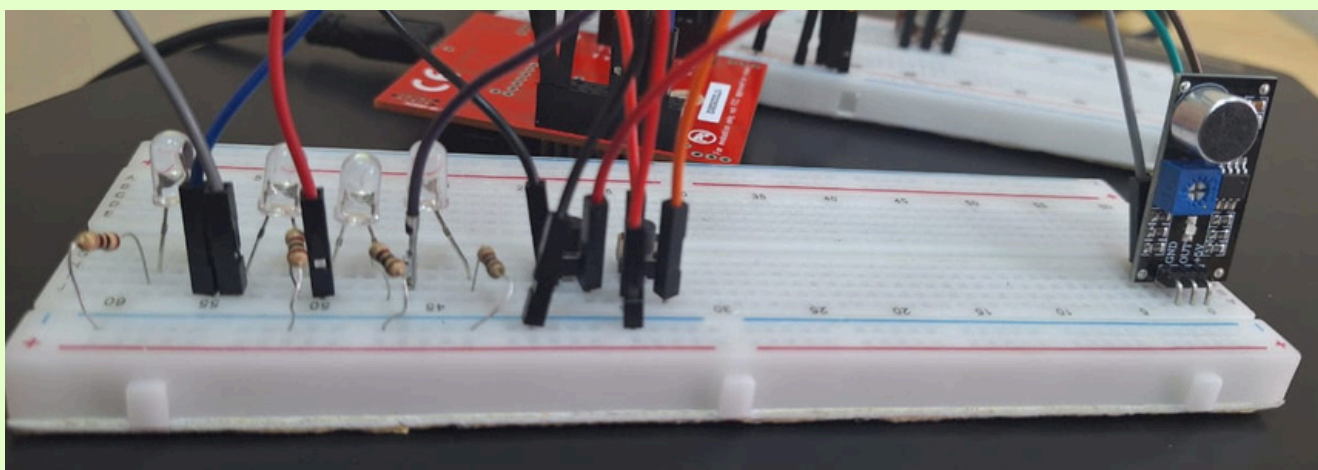
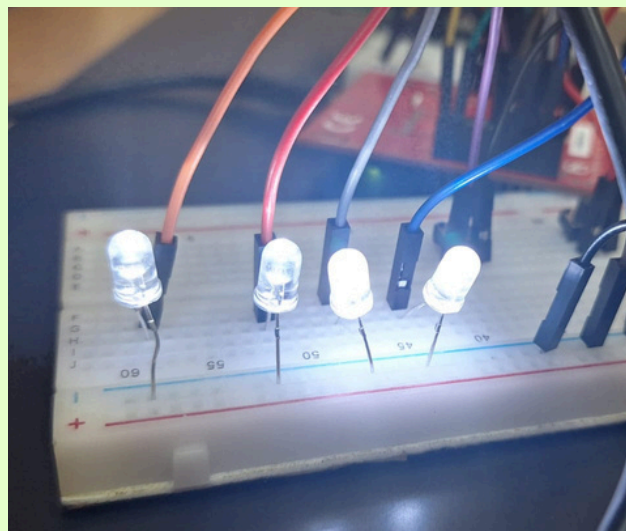
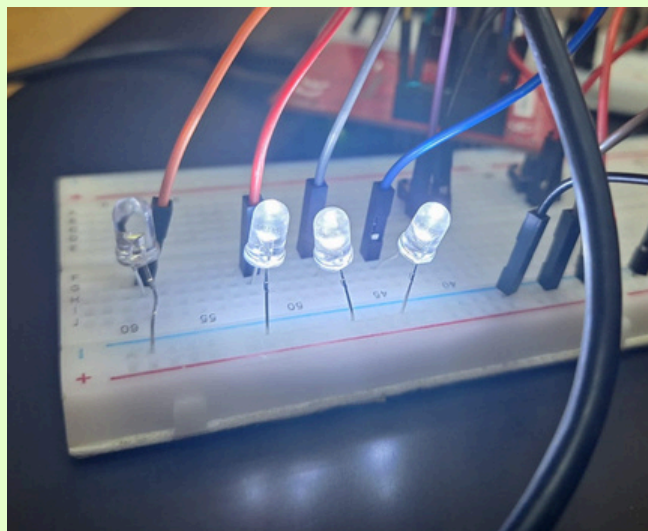
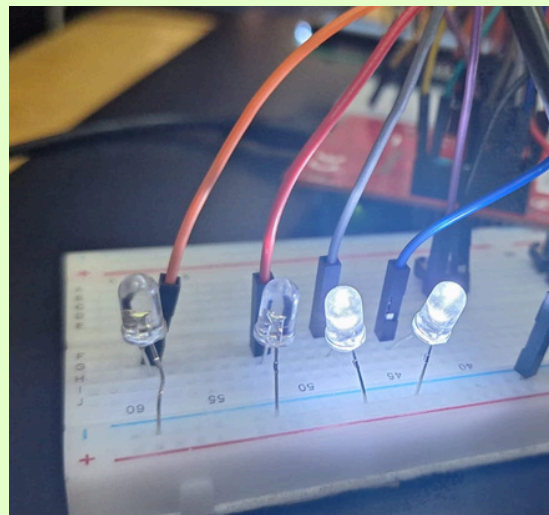
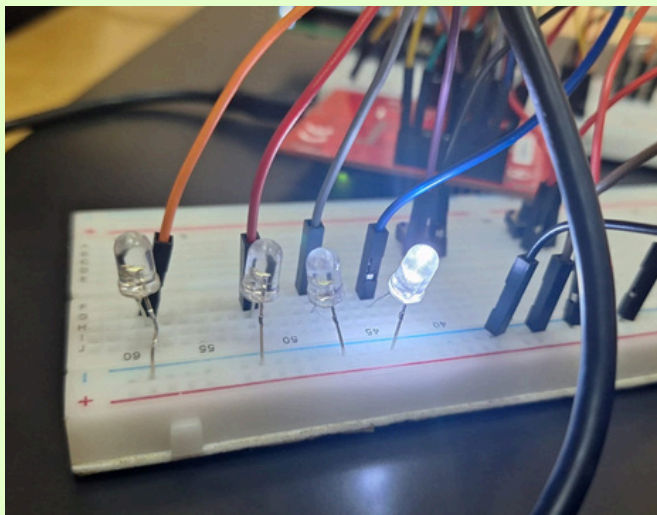
1. `GPIO_Init(port, pin, INPUT, DIGITAL)`
2. `GPIO_SetDirection(port, pin, INPUT)`
3. `GPIO_EnablePullUp(port, pin)`

2. BUTTON_charRead

This function reads the current state of the button.

- `GPIO_ReadPin(port, pin)`

CIRCUIT



CIRCUIT

